

FINAL YEAR PROJECT BRIEFING

Technical Feasibility & Planning Document

Website Audit AI

AI-Powered Website Analysis & Improvement System

STUDENTS

Muhammad Masab — BSCS51F23S006

Muzzain Abbas — BSCS51F23S015

SUPERVISOR

Dr. Hassam Ali

DEPARTMENT

Computer Science — BS Final Year, 2023–2027

VERSION

v1.0 — September 2026

SECTION 01

Project Summary & Naming

Product names, company identity, and a concise summary of what this project actually is.

1.1 What This Project Is

You are building a web-based AI-powered tool that accepts a website URL or screenshot, analyses the site for UI/UX quality, content effectiveness, and technical SEO signals, and generates a structured improvement report. The system uses a combination of rule-based code analysis and large language model APIs. It is not a full-scale SaaS platform – it is a demonstrable academic prototype with clearly defined scope.

HONEST ASSESSMENT

Your vision document describes something closer to a \$2M funded startup product. What two final-year students can realistically build in 6–8 months is a focused, demonstrable core version of one part of that vision. This document defines exactly what that is.

1.2 Recommended Product Names

NAME	WHY IT WORKS	VERDICT
Auditly	Clear function, modern -ly suffix common in SaaS. Easy to say and remember.	✔Recommended
SiteIQ	Short, professional. "IQ" implies intelligence and scoring. Broad enough to grow.	✔Recommended – Top Pick
PagePulse	Alliterative, implies monitoring and vitals. Strong SaaS feel.	✔Recommended
Audita	Clean, distinctive. Sounds like a proper brand name.	Consider
WebLens	Metaphor of looking closely at a site. Clear and visual.	Consider

Primary recommendation: SiteIQ – short, implies intelligence, works as siteiq.app, broad enough for future expansion. For your FYP, name the product SiteIQ and clarify in documentation that this is an academic prototype.

1.3 Recommended Company Names

NAME	VISION	VERDICT
Axiom Labs	Self-evident truths, precision tools. Clean and technical.	✔Top Pick
Verno	Very short invented word, memorable, easy to trademark.	✔Top Pick
LumiQ	Lumen + IQ. Clarity and intelligence.	Good
Quorient	Quotient + orient. Smart-sounding but slightly hard to spell.	Risky

Recommendation: Axiom Labs. Product becomes: SiteIQ by Axiom Labs.

1.4 Logo Concept Ideas

All concepts follow the principle: minimal, geometric, meaning-bearing. The logo must work at 16px (favicon) and 300px equally.

- Concept A — The Lens: A thin circular ring with a small crosshair at centre. Suggests inspection, focus, precision. Single colour: Navy #0A1628. Works as monogram with "S" inside the circle.
- Concept B — The Pulse Line: A flat horizontal line that rises into a sharp peak and returns flat. Suggests vitals, heartbeat, real-time analysis.
- Concept C — Stacked Bars: Three short horizontal bars of decreasing length (like a chart/score). Suggests audit, grading, analytics. Very legible at small sizes.
- Concept D — The Window Frame: A minimal browser window outline. Immediately communicates "website" without being literal.

Recommendation: Concept A (The Lens) or Concept C (Stacked Bars). Both are buildable in Figma in under an hour. Avoid gradients, multiple colours, or complex illustrations.

SECTION 02

Critical Honest Assessment

This section intentionally challenges your assumptions. You asked for an honest, critical analysis — this is it.

IMPORTANT NOTICE

The original vision document describes 22+ major features, 4 distinct analysis categories, AI-generated wireframes, SEO analysis, performance analysis, accessibility audits, competitor analysis, mobile audits, and a complete report generation engine. That is not one FYP. That is a team of 8 engineers working for 18 months.

2.1 What Is Strong About the Idea

- The core concept is genuinely useful. Businesses pay real money for website audits. The problem is valid.
- Using LLMs for qualitative analysis is the right approach. This is exactly the kind of judgment-based task that benefits from AI.
- The screenshot input method is technically feasible and more reliable than URL scraping for visual analysis.
- The context questionnaire is excellent — asking users about their target audience and goals genuinely improves AI output quality. This is one of the best ideas in the proposal.
- The structured JSON output approach is correct. You have understood how to get reliable data from LLMs.

2.2 Where You Are Wrong or Overly Optimistic

Problem 1 — "AI will analyse visual hierarchy from a URL"

This is only partially true, and harder than it sounds. When you fetch a URL you get HTML, CSS, and DOM structure — you do not get how the page actually looks rendered. A CSS file might declare font-size: 24px but the element might be hidden or only visible after a JavaScript interaction. To actually see the rendered page you need a headless browser (Playwright or Puppeteer), which is a significant technical component to build, debug, and maintain. You are not simply making an HTTP request.

Problem 2 — "The AI can generate wireframes"

You cannot generate actual visual wireframes using a text-based LLM API. You would need a diffusion model (like Stable Diffusion) plus fine-tuning, or a specialised tool. For an FYP, this is completely out of scope. What you can do is have the AI describe layout changes in structured text rendered as ASCII-style representations. This is achievable. Actual visual wireframe generation is not.

Problem 3 — "We can score everything out of 100"

A score only has meaning if it has a documented, reproducible methodology. If your system says "UI/UX: 72/100" and the score changes every time you run the same website because the AI gives a different response, that score is meaningless — and academically

indefensible. You either define a deterministic scoring formula or you accept that LLM-influenced scores will have variance, which you must document. Arbitrary AI scores presented as objective metrics will be criticised by your supervisor.

Problem 4 — "We will analyse mobile responsiveness"

True mobile responsiveness testing requires either a real device farm (costs money) or emulation through a headless browser at different viewport sizes. This is possible but adds significant complexity. For the MVP, limit this to checking meta viewport tags, CSS media query presence, and maximum page width — not full visual rendering.

Problem 5 — "Website URL analysis is straightforward"

Approximately 30–40% of the web is protected behind Cloudflare or other bot protection. JavaScript-heavy SPAs will return nearly empty HTML to a simple HTTP request. Your scraper will fail on many modern websites unless you use a headless browser, which comes with its own challenges.

SECTION 03

Refined Scope & MVP Definition

After critical analysis, this section defines exactly what Version 1 (your actual FYP) should contain, and what should be labelled as future work.

3.1 The Correct Mental Model

Stop thinking of this as a "website audit tool." Start thinking of it as a "Screenshot + Context → AI Audit Report Generator." That framing immediately makes the scope clear and achievable. You accept a screenshot (or optionally a URL for basic HTML extraction), combine it with user-provided context, send it to an AI vision model, and return a structured professional audit report.

RECOMMENDED MVP DEFINITION

A web application where a user uploads a full-page website screenshot, fills in a context form about their business and audience, and receives a structured AI-generated audit report covering UI/UX observations, copy quality, and actionable improvement suggestions, displayed in a readable dashboard and exportable as PDF.

3.2 MVP Features (Build These)

FEATURE	INPUT REQUIRED	OUTPUT	FEASIBILITY
User authentication (register/login)	—	Secure session	Easy
Screenshot upload	PNG/JPG file	Stored image	Easy
Business context form	Text inputs	Structured JSON	Easy
URL input →basic HTML extraction	URL string	DOM text, meta, headings	Moderate
AI vision analysis (screenshot)	Image + context	Structured JSON findings	Moderate
AI copy analysis	Text content	Copy improvement suggestions	Moderate
Deterministic checks (SEO, meta, headings)	HTML/DOM	Pass/fail findings	Moderate
Structured findings display (dashboard)	AI + rule output	Categorised findings UI	Moderate
Priority scoring (evidence-based)	Findings data	Scored categories	Hard
Audit history (past audits)	User account	List + retrieval	Easy
PDF export of report	Findings data	Downloaded PDF	Hard

3.3 Future Features (Do NOT Build in MVP)

FEATURE	WHY EXCLUDED FROM MVP
AI-generated visual wireframes	Requires diffusion models — outside scope entirely
Competitor analysis	Requires crawling multiple sites — too broad

FEATURE	WHY EXCLUDED FROM MVP
Full mobile responsiveness testing	Requires headless browser at multiple viewports
Performance audit (Core Web Vitals)	Use Google PageSpeed API in V2 — do not reinvent this
Accessibility full audit (WCAG 2.1)	axe-core library can provide this in V2
Audit comparison between two versions	V2 feature once core auditing is stable
Team collaboration features	SaaS platform feature, not FYP scope
JavaScript-heavy SPA crawling	Playwright integration is complex for beginners — defer to V2

SECTION 04

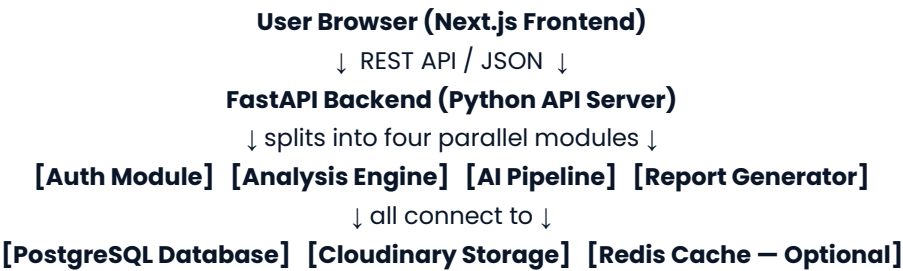
System Architecture

The complete technical architecture of the system. Every component is explained. This diagram represents the MVP only.

4.1 Component Responsibilities

COMPONENT	TECHNOLOGY	RESPONSIBILITY
Frontend	Next.js 14 + Tailwind CSS	User interface: upload, form, dashboard, report viewer
Backend API	FastAPI (Python)	Orchestrates all analysis steps, serves results to frontend
Auth Module	NextAuth.js + JWT	User registration, login, session management
Scraper/Extractor	BeautifulSoup + httpx	Basic HTML extraction from URL (non-JS sites)
Rule Engine	Custom Python	Deterministic checks: meta tags, headings, alt text, links
AI Pipeline	Claude API (claude-sonnet-4-5)	Vision analysis + copy analysis + recommendations
Report Generator	WeasyPrint or ReportLab	Convert structured findings to downloadable PDF
Database	PostgreSQL + Prisma ORM	Store users, audit records, findings, scores
File Storage	Cloudinary (free tier)	Screenshot image storage and retrieval
Deployment	Vercel (frontend) + Railway (backend)	Hosting and CI/CD pipeline

4.2 Architecture Flow (Text Representation)



4.3 How Components Communicate

The frontend communicates exclusively with the backend via REST API calls over HTTPS. The backend is the only component that ever calls the Claude API — the frontend never touches AI credentials. All screenshots are uploaded directly to Cloudinary via a signed upload URL. The backend never stores image files on disk.

SECTION 05

Input Methods: URL vs Screenshot

A detailed technical comparison of both input methods — their capabilities, limitations, and what can realistically be achieved.

5.1 Comparative Analysis

CAPABILITY	URL ANALYSIS	SCREENSHOT ANALYSIS
Visual hierarchy detection	✗Cannot see rendered output	✓✓Vision model sees what users see
Typography appearance	△CSS values only, not visual result	✓✓Actual font rendering visible
Layout composition	✗Cannot determine without rendering	✓✓Vision model evaluates layout
Meta description check	✓✓Direct HTML extraction	✗Not visible in screenshot
Heading hierarchy (H1–H6)	✓✓DOM extraction	△Inferable but not reliable
Alt text on images	✓✓HTML attribute check	✗Not extractable from image
Broken links	✓✓HTTP requests to href values	✗Not testable from screenshot
JavaScript-heavy sites	✗Simple fetch returns empty HTML	✓✓Screenshot captures final render
Cloudflare-protected sites	✗Will be blocked	✓✓User provides screenshot themselves
CTA visibility and placement	✗No visual context	✓✓Vision model evaluates
Whitespace and spacing	✗CSS values ≠ perceived spacing	✓✓Visible in screenshot

5.2 Architecture Decision

RECOMMENDATION

Primary input: Screenshot upload. This is more reliable, works on all websites regardless of JS framework or bot protection, and is simpler to implement. Secondary input: URL (optional, for HTML extraction only). When a URL is provided, the system attempts a simple HTTP fetch. If this fails, the system falls back gracefully to screenshot-only analysis.

5.3 What You Can Extract from a Simple HTTP Request

CAN EXTRACT: Page title and meta description, heading tags (H1–H6) and content, all body text (static sites only), image alt text attributes, internal and external links, OG tags, canonical tag, Schema markup, robots meta tag, viewport meta tag, CSS and JS file references.

CANNOT EXTRACT WITHOUT RENDERING: Content generated by JavaScript frameworks, actual rendered layout and visual composition, real typography appearance, perceived

colour contrast, element visibility states, overlay content, animation elements, sticky header behaviour, scroll-triggered content, actual page rendering on mobile.

SECTION 06

Deterministic vs AI Analysis

Some checks should never use AI — they are faster, cheaper, and more accurate using traditional code. Others cannot be done by code at all.

6.1 Division of Responsibility

TASK	TRADITIONAL CODE	LLM	VISION	EXTERNAL API
Missing meta description	✓Primary	—	—	—
Missing H1 tag	✓Primary	—	—	—
Multiple H1 tags	✓Primary	—	—	—
H1–H2–H3 hierarchy violation	✓Primary	—	—	—
Images missing alt text	✓Primary	—	—	—
Missing viewport meta tag	✓Primary	—	—	—
No HTTPS	✓Primary	—	—	—
Title tag length (0–60 chars)	✓Primary	—	—	—
Meta description length	✓Primary	—	—	—
Page load / Core Web Vitals	—	—	—	✓PageSpeed API (v2)
Heading copy quality	—	✓Primary	—	—
CTA copy analysis	—	✓Primary	—	—
Body copy clarity	—	✓Primary	—	—
Improved copy suggestions	—	✓Primary	—	—
Visual hierarchy assessment	—	—	✓Primary	—
CTA placement and visibility	—	—	✓Primary	—
Layout composition quality	—	—	✓Primary	—
Whitespace usage	—	—	✓Primary	—
Trust signal assessment	—	✓Partial	✓Primary	—
UX recommendations	—	✓Primary	✓Input	—

6.2 Why This Division Matters

Using an LLM to check whether a meta description exists is wasteful, slow, and introduces unnecessary cost. The API call takes 2–5 seconds and costs money for a task a single line of Python can complete in milliseconds. Conversely, asking rule-based code to evaluate whether a headline is persuasive is impossible — there is no deterministic formula for copywriting quality.

SECTION 07

AI Integration Strategy

A critical distinction: AI used to build this project is completely different from AI integrated into the product itself.

7.1 Two Types of AI Usage

CATEGORY	AI FOR DEVELOPMENT	AI IN THE PRODUCT
What it is	Claude/Cursor/Copilot used to write code	Claude API called by backend at runtime
Who triggers it	You, the developer	Your users when they submit an audit
Is it a product feature?	No — development tool only	Yes — this is the FYP's AI integration
Does it count in FYP?	No	Yes
Cost	~\$20/month Cursor subscription	~\$0.01–\$0.15 per audit call

7.2 Recommended AI Model

MODEL RECOMMENDATION

Use claude-sonnet-4-5 for product integration. It balances cost and quality well for this use case. Avoid using the most expensive model (Opus) for every call. Always verify the exact model string at docs.anthropic.com at the time of integration, as model names are updated periodically.

7.3 Three AI Calls Per Audit

CALL	MODEL TYPE	INPUT	OUTPUT	APPROX. COST
Call 1 — Vision Audit	Claude (Vision)	Screenshot + user context + system prompt	JSON: visual findings array	\$0.05–\$0.12
Call 2 — Copy Analysis	Claude (Text)	Extracted text + context + instructions	JSON: copy issues + rewrites	\$0.01–\$0.04
Call 3 — Recommendations	Claude (Text)	All findings from calls 1+2 + rules output	JSON: prioritised action plan	\$0.01–\$0.03

7.4 Handling Hallucinations

AI models sometimes confidently produce incorrect information. Mitigate this in three ways: (1) instruct the model to only comment on elements it can clearly see in the screenshot or text received; (2) ask for a confidence level (high/medium/low) with each finding; (3) never present AI findings as objective facts — label them clearly as "AI Analysis" in your UI. Deterministic findings can be presented as objective facts because they are.

SECTION 08

AI Prompt Architecture

Prompt engineering is designing a reliable data pipeline. Structured JSON outputs are the foundation of a predictable, maintainable system.

8.1 Why JSON Output Is Essential

If you simply ask the AI "analyse this website and tell me what's wrong," you receive free-text that changes structure every time. You cannot display that reliably in a dashboard, store it in a database, or use it to calculate scores. Instruct the model to return structured JSON matching your database schema. The model should return only JSON — no preamble, no markdown, no explanation outside the JSON structure.

8.2 Prompt Structure for Vision Audit

SYSTEM PROMPT:

You are a professional UX/UI auditor. Analyse the provided screenshot and return ONLY valid JSON matching the schema. Do not include markdown, explanations, or any text outside the JSON.

USER MESSAGE:

[IMAGE: base64-encoded screenshot]
Business context: {business_type}, {target_audience}, {primary_goal}, {desired_action}
Instructions: Identify specific UI/UX issues visible in this screenshot. For each issue provide the required JSON fields. Limit your response to the 8 most significant issues.

8.3 Finding JSON Schema

FIELD	TYPE	DESCRIPTION
id	String	Unique identifier (e.g. "finding_001")
category	Enum	visual_hierarchy typography color layout cta navigation whitespace trust copy
severity	Enum	critical high medium low
element	String	Affected section (e.g. "hero_section")
problem	String	Clear description of what is wrong
evidence	String	What the model specifically observed in the screenshot
impact	String	How this affects the user or business goal
recommendation	String	Specific, actionable improvement suggestion
design_principle	String	Design principle cited (e.g. "F-Pattern, Visual Hierarchy")
confidence	Enum	high medium low — model's confidence in this finding

8.4 Validation Before Storage

After receiving JSON from the AI, your backend must validate it: (1) is the response valid JSON? (2) does it contain required fields? (3) are enum values within expected ranges? If

validation fails, retry once with an error message in the prompt. Do not store malformed data — it will corrupt report generation later.

SECTION 09

Scoring Methodology

Scores are only academically credible if they have a documented, reproducible methodology.

CRITICAL WARNING

Do NOT ask the AI to return a score out of 100. The score will change each time you run the same website because LLM outputs are non-deterministic. An AI score without a documented formula is meaningless and will not survive supervisor scrutiny. Use the methodology below instead.

9.1 Score Categories

CATEGORY	WEIGHT	METHODOLOGY	SOURCE
Technical SEO	20%	100% deterministic. Each of 10 checks is pass/fail with defined point values.	Rule engine
Accessibility Basics	15%	Deterministic: alt text, viewport, heading order. Each violation deducts fixed points.	Rule engine
UI/UX Quality	30%	AI findings: severity weighting (critical=-15, high=-8, medium=-4, low=-1). Max 8 findings.	AI Vision
Copy Effectiveness	25%	AI findings: same severity deduction formula. Starts at 100, deductions applied per finding.	AI Text
Conversion Readiness	10%	Hybrid: CTA presence (deterministic) + CTA visibility (AI vision) + CTA copy quality (AI text).	Hybrid

9.2 Technical SEO Scoring Example

CHECK	POINTS	PASS/FAIL CRITERIA
H1 tag present (exactly one)	15 pts	PASS: exactly 1 H1. FAIL: 0 or more than 1.
Meta description present	15 pts	PASS: present and non-empty. FAIL: missing or empty.
Meta description length	10 pts	PASS: 140-160 chars. PARTIAL (5 pts): 100-139 or 161-180. FAIL: outside range.
Title tag present and length	15 pts	PASS: present and 10-60 chars. FAIL: missing or outside range.
Viewport meta tag present	10 pts	PASS: present. FAIL: missing.
Images have alt text	15 pts	PASS: 100%. PARTIAL: -1pt per missing alt text. FAIL: below 50%.
Canonical tag	5 pts	PASS: present. FAIL: missing.
OG title and description	5 pts	PASS: both present. PARTIAL (2.5 pts): one present. FAIL: both missing.
Schema markup present	5 pts	PASS: valid schema. PARTIAL (2.5 pts): malformed. FAIL: absent.
HTTPS enforced	5 pts	PASS: HTTPS. FAIL: HTTP.

9.3 AI-Influenced Score Disclosure

When displaying scores that include AI input, your UI must clearly label them as "AI-Assisted Score" and note that they may vary slightly between analyses. Include a brief tooltip or footnote explaining the scoring methodology. This strengthens the academic credibility of your project significantly.

SECTION 10

Data Model & Database Design

The database must store user accounts, audit submissions, AI findings, rule-based findings, scores, and generated reports.

10.1 Entity Descriptions

ENTITY	KEY FIELDS	RELATIONSHIPS
USER	id, email, password_hash, name, created_at, audit_count	One user —many audits
AUDIT	id, user_id, website_url, screenshot_url, status, created_at, completed_at	Belongs to USER —has WEBSITE_CONTEXT, many FINDINGS, many SCOREs
WEBSITE_CONTEXT	id, audit_id, business_type, target_audience, primary_goal, desired_action, competitors	One-to-one with AUDIT
FINDING	id, audit_id, source (ai_vision/ai_text/rule_based), category, severity, element, problem, evidence, impact, recommendation, confidence	Many per AUDIT
SCORE	id, audit_id, category, raw_score, weight, methodology (deterministic/ai_assisted)	Many per AUDIT (one per category)
COPY_SUGGESTION	id, audit_id, element_type, original_text, issue, improved_text, reason	Many per AUDIT

10.2 Audit Status Flow

Every audit passes through these states: pending (record created, processing not yet started) →processing (analysis pipeline is running) →complete (all findings stored, scores calculated) →failed (a critical error occurred, partial results saved). The frontend polls the /api/audits/{id}/status endpoint every 3 seconds until the status changes from "processing."

SECTION 11

Recommended Technology Stack

Every technology selected for three reasons: it works well for this specific problem, has excellent documentation, and is learnable in a reasonable timeframe.

LAYER	TECHNOLOGY	WHY THIS CHOICE	ALTERNATIVE
Frontend Framework	Next.js 14 (React)	SSR, file-based routing, huge community, free Vercel deployment. Most modern SaaS products use it.	Create React App
CSS Styling	Tailwind CSS	Utility-first, minimal config, fast development, pairs perfectly with Next.js.	CSS Modules
UI Components	shadcn/ui	Pre-built accessible components (buttons, forms, cards, tables). Works with Tailwind. Copy-paste, no heavy dependency.	Radix UI / Chakra UI
Charts / Dashboard	Recharts	React-native charting, covers bar, radar, and pie charts needed for score display.	Chart.js
Backend Framework	FastAPI (Python)	Python is better suited for AI integration and HTML parsing. Modern, fast, auto-generates API docs.	Express.js (Node)
HTML Extraction	htpx + BeautifulSoup4	htpx is async-capable. BS4 is the industry standard for HTML parsing. Simple to learn.	requests + lxml
AI API	Anthropic Claude API	Supports vision, produces reliable JSON, excellent documentation.	OpenAI GPT-4o
Database	PostgreSQL	Relational with proper JSON support. Free on Supabase or Railway.	SQLite (too limited)
ORM	Prisma (frontend) + SQLAlchemy (backend)	Prisma for type-safe DB access in Next.js. SQLAlchemy for Python backend.	Drizzle ORM
Authentication	NextAuth.js v5	Handles JWT sessions, OAuth providers, credentials auth. Integrates into Next.js.	Auth0
File Storage	Cloudinary (free tier)	25 GB free storage, handles image optimisation, simple upload API.	Supabase Storage
PDF Generation	WeasyPrint (Python)	Converts HTML/CSS to PDF. Design report in HTML (which you know) and export.	ReportLab
Frontend Deployment	Vercel	Free for academic projects, one-click Next.js deployment, automatic preview deployments.	Netlify
Backend Deployment	Railway	\$5/month credit (free with student plan). Deploys Python apps easily.	Render.com
Version Control	Git + GitHub	Non-negotiable. Two repositories (frontend, backend) with clear commit history.	—

SECTION 12

IDE & Development Environment

You asked specifically whether to buy Claude Pro (\$20/month) or Cursor (\$20/month). This section gives you a direct answer.

12.1 Tool Comparison

TOOL	COST	PRIMARY STRENGTH	BEST FOR
Cursor	\$20/mo	AI code completion inside editor, understands your entire codebase, Composer mode for multi-file edits	Primary coding tool — recommended for this project
VS Code	Free	Largest extension ecosystem, Git integration, best Python/JS support	Fallback or pairing with Claude.ai
Claude.ai Pro	\$20/mo	Longer context windows, file uploads, deeper analysis	Architecture planning, debugging complex logic
Windsurf	Free (limited)	Similar to Cursor, Cascade AI mode	Optional alternative to Cursor
GitHub Copilot	Free for students	In-editor autocomplete, integrates into VS Code	Good free option if student account available

12.2 Direct Recommendation

RECOMMENDATION: CURSOR

Get Cursor (\$20/month) as your primary coding tool. Cursor understands your entire project, can suggest fixes across multiple files simultaneously, and dramatically speeds up development for beginners. This is more valuable than Claude.ai Pro during the active coding phase because it lives inside your editor. If you can only afford one: Cursor. Also check GitHub Student Developer Pack — it includes GitHub Copilot for free.

12.3 How to Use AI Coding Tools Properly

- 1. Plan first, code second. Discuss architecture and data flow before writing a single line of code.
- 2. One module at a time. Ask AI to build one function or component at a time and explain each line. If you cannot explain what the code does, you do not understand it — and your supervisor will know.
- 3. Always read the generated code. AI produces bugs. It sometimes uses deprecated APIs or makes incorrect assumptions about your project structure.
- 4. Test each module before moving on. When one component works, commit it. Then build the next.
- 5. Write commit messages that explain your work ("Added screenshot upload handler" — not "wip").
- 6. Use AI for debugging first, Stack Overflow second.
- 7. Never let AI write your FYP report or documentation. Your supervisor can tell. Your own technical writing, even if imperfect, is far more credible.

SECTION 13

Required Skills & Learning Roadmap

Honest skills assessment divided into what you must know before starting, what to learn during development, and what is optional.

Level 1 — Must Know Before Starting (Weeks 1–4)

These are non-negotiable prerequisites. You cannot build the first feature without them.

- HTML fundamentals
- CSS basics + Flexbox/Grid
- JavaScript ES6+ (let, const, arrow functions, async/await)
- React fundamentals (components, props, state, hooks)
- REST API concepts (GET, POST, JSON, status codes)
- Git + GitHub (commit, push, pull, branch)
- Python basics (functions, dicts, lists, loops)
- Terminal / command line basics
- npm / pip package installation

Level 2 — Learn During Development (Weeks 4–16)

You do not need to master these before starting. Learn them as each feature requires it.

- Next.js 14 (routing, API routes, Server Components)
- Tailwind CSS utility classes
- FastAPI (endpoints, Pydantic models, async handlers)
- PostgreSQL basics (tables, queries, joins)
- SQLAlchemy ORM models
- HTTP requests in Python (httpx)
- BeautifulSoup HTML parsing
- JSON handling and validation
- Environment variables (.env)
- Anthropic Claude API integration
- Basic prompt engineering
- Authentication with NextAuth.js
- File upload handling
- Cloudinary API
- Deployment on Vercel + Railway
- Basic TypeScript types

Level 3 — Advanced / Optional

- Playwright headless browser
- Redis job queues
- WCAG accessibility standards
- Google PageSpeed API
- WeasyPrint PDF generation

- Docker containerisation
- Unit testing (pytest, Jest)
- CI/CD pipelines

13.1 Team Skill Division

AREA	MUHAMMAD MASAB	MUZZAIN ABBAS	BOTH
Frontend (Next.js, React, Tailwind)	✔Lead	Support	—
Backend (FastAPI, Python)	Support	✔Lead	—
AI Integration (Claude API)	Support	✔Lead	—
HTML Extraction (BS4)	—	✔Lead	—
Database Design	—	—	✔Both
API Contract Design	—	—	✔Both
Report Design (HTML template)	✔Lead	—	—
Testing	—	—	✔Both
FYP Documentation	—	—	✔Both
Deployment	Frontend (Vercel)	Backend (Railway)	—

SECTION 14

Prerequisites & Cost Breakdown

Everything you need before writing the first line of code, and an honest breakdown of what this project will cost.

14.1 Software & Accounts Required

REQUIREMENT	TOOL/PLATFORM	COST	PURPOSE
Code editor	Cursor	\$20/mo (1 licence, share)	AI-assisted development
Node.js runtime	nodejs.org	Free	Running Next.js
Python	python.org	Free	Backend development
Git	git-scm.com	Free	Version control
GitHub accounts (×2)	github.com	Free	Repository hosting
Vercel account	vercel.com	Free (Hobby plan)	Frontend deployment
Railway account	railway.app	\$5/mo credit (often free)	Backend deployment
Supabase account	supabase.com	Free (500 MB DB)	PostgreSQL database
Cloudinary account	cloudinary.com	Free (25 GB)	Screenshot storage
Anthropic API key	console.anthropic.com	Pay-per-use (~\$6–16 total)	AI analysis in product
Postman / Insomnia	postman.com	Free	API testing
Figma	figma.com	Free (Education)	UI wireframing

14.2 Total Project Budget

ITEM	MONTHLY	6-MONTH TOTAL
Cursor (1 licence, split between 2 students)	\$20	\$120
Railway backend hosting	~\$0–\$5	~\$0–\$30
Claude API credits	~\$2–\$3 average	~\$12–\$18 total
All other services	\$0	\$0
TOTAL	—	~\$132–\$168 (split two ways)

This is approximately PKR 6,000–7,500 each over the entire project duration — a reasonable budget for a final year project. If cost is a concern, check the GitHub Student Developer Pack for potential free Railway credits.

SECTION 15

Weaknesses & Technical Risks

Every risk documented with its cause, impact, and realistic mitigation. Acknowledging weaknesses in your FYP documentation is a sign of maturity.

RISK	LIKELIHOOD	IMPACT	MITIGATION
AI hallucination in visual analysis	High	High	Label AI findings clearly. Include confidence levels. Instruct model to cite specific visual evidence. Document as inherent characteristic.
URL scraping fails on JS-heavy sites	High	Medium	Detect JS-heavy sites and fall back to screenshot-only analysis with a clear user message.
Subjectivity of design evaluation	Medium	Medium	Ground AI analysis in established design principles. Require the model to cite a principle per finding.
Scope creep during development	High	High	Agree on fixed feature list with supervisor by Week 6. No new features after Week 12.
Long processing times (40+ seconds)	Medium	Medium	Implement real-time progress indicator with step descriptions. Use async polling pattern.
SSRF vulnerability in URL input	Low	High	Validate all submitted URLs: only HTTP/HTTPS; block private IP ranges; verify domain resolves to public IP.
Prompt injection through website content	Low	Medium	Sanitise extracted text before including in prompts. Instruct model to ignore instructions found in page content.
Deployment fails at demo time	Low	High	Deploy to production by Week 16. Test on live deployment weekly from that point.

SECTION 16

Security & Legal Considerations

Not legal advice — these are technical considerations that should be addressed in your system design and FYP report.

16.1 Security Requirements

RISK AREA	REQUIREMENT	IMPLEMENTATION
API key exposure	Never commit API keys to GitHub	.env files + platform secret management
Authentication	Password hashing, secure sessions	bcrypt for passwords, JWT with expiry
URL input validation	SSRF prevention	Block private IP ranges, validate scheme
File upload security	Validate file type and size	Accept only PNG/JPG up to 10 MB, verify MIME type
Prompt injection	Sanitise web content before prompting	Strip scripts, encode special characters
Rate limiting	Prevent API abuse	Max 5 audits per user per day
Data exposure	Users see only their own audits	Enforce user_id filter on all audit queries
HTTPS everywhere	All traffic encrypted	Vercel and Railway enforce HTTPS automatically

16.2 Website Crawling

When your system fetches a website URL, it acts as a web crawler. Robots.txt is a convention — not a legal requirement — but violating it may violate a website's Terms of Service. For your academic project, document that users are responsible for only submitting URLs they have permission to analyse. Screenshot upload avoids this issue entirely.

16.3 Prompt Injection

If a malicious website owner embeds instructions in their page's hidden text, your system might pass that content to the AI model. Mitigate this by: sanitising all extracted text before including it in prompts; instructing the model in the system prompt to ignore any instructions found in page content; and validating AI output schema to ensure it conforms to expected structure.

16.4 AI-Generated Recommendations Disclaimer

Your audit report constitutes suggestions, not professional design advice. Include a brief disclaimer in the report output stating that recommendations are AI-generated and should be reviewed by a design professional before implementation.

SECTION 17

Feature Priority Matrix

Every feature rated by difficulty, importance, and whether it belongs in the MVP.

FEATURE	DIFFICULTY	IMPORTANCE	AI?	VERSION
User registration / login	Low	Critical	No	MVP
Screenshot upload	Low	Critical	No	MVP
Business context form	Low	High	No	MVP
URL →HTML extraction	Medium	High	No	MVP
Deterministic SEO checks	Medium	High	No	MVP
AI vision audit (screenshot)	Hard	Critical	Yes (Vision)	MVP
AI copy analysis	Medium	High	Yes (Text)	MVP
AI recommendations generation	Medium	High	Yes (Text)	MVP
Score calculation engine	Medium	High	Partial	MVP
Findings dashboard (display)	Medium	Critical	No	MVP
Audit history list	Low	Medium	No	MVP
PDF export	Hard	Medium	No	MVP
Progress indicator during analysis	Medium	High	No	MVP
Headless browser (full rendering)	Very Hard	Medium	No	V2
Google PageSpeed API integration	Low	Medium	No	V2
Full WCAG accessibility audit	Hard	Medium	No	V2
Mobile viewport emulation	Very Hard	Medium	No	V2
Audit comparison (two versions)	Hard	Low	No	V2
Competitor analysis	Very Hard	Low	Yes	V3
AI wireframe generation	Very Hard	Low	Yes (Diffusion)	V3

SECTION 18

Development Roadmap

A realistic 20-week development plan for two students. Phases are sequenced to build testable, demonstrable functionality at each milestone.

PHASE 01	Research, Requirements & Architecture	Weeks 1–3
Define the refined scope formally. Study all technologies. Set up development environments. Design the database schema. Define the API contract between frontend and backend. Create wireframes for all key pages. Deliverable: Architecture document + wireframes + DB schema + signed GitHub repos		
PHASE 02	Foundation — Auth, DB, Basic Frontend	Weeks 4–6
Build authentication (register, login, session), database tables via Prisma migrations, and the basic Next.js page structure with navigation. Deliverable: Working login/register flow, empty dashboard, connected to Supabase		
PHASE 03	Input — Screenshot Upload & Context Form	Weeks 7–8
Build the screenshot upload UI and backend handler. Store images in Cloudinary. Build the business context form. Store submissions in the AUDIT and WEBSITE_CONTEXT tables. Deliverable: User can upload screenshot and fill context form — data stored in DB		
PHASE 04	Analysis Engine — Deterministic Rules	Weeks 9–10
Build the Python rule engine: URL fetching with httpx, HTML parsing with BS4, and all deterministic checks (meta, headings, alt text, links, schema). Deliverable: Backend returns structured rule-based findings for any URL		
PHASE 05	AI Integration — Vision & Copy Analysis	Weeks 11–13
Build the three AI calls: vision audit, copy analysis, and recommendation generation. Implement prompt templates, JSON validation, and error handling. This is the most technically challenging phase. Deliverable: Full audit pipeline returns structured findings from all three sources		
PHASE 06	Dashboard, Scoring & Report Display	Weeks 14–15
Build the audit results dashboard: categorised findings, severity badges, score cards, copy suggestions, and recommendations section. Implement the scoring formula. Deliverable: Complete, readable audit report displayed in the web application		

PHASE 07 **PDF Export & Polish** Weeks 16–17

Build PDF export using WeasyPrint. Refine all UI components. Add progress indicator. Complete audit history page. End-to-end testing on 5–10 websites.

Deliverable: Fully working system demonstrable end-to-end

PHASE 08 **Evaluation & Documentation** Weeks 18–20

Conduct formal evaluation on 15–20 websites. Write FYP documentation. Prepare presentation. Final deployment and system stability check.

Deliverable: Deployed system + FYP documentation + evaluation results + presentation

SECTION 19

Testing & Evaluation Methodology

A proper evaluation methodology is what distinguishes an academic FYP from a hobby project.

19.1 Two Categories of Testing

Testing this project requires two fundamentally different approaches: deterministic testing for rule-based checks (automated, objective, pass/fail), and qualitative evaluation for AI-generated findings (manual, requires human judgment).

19.2 AI Findings Evaluation Procedure

- 1. Select 15–20 test websites across categories: eCommerce, SaaS, portfolio, blog, service business, non-profit. Include 2–3 known "bad" and 2–3 "good" websites for contrast.
- 2. Manually audit each website before running your system. Document your own findings. This creates the ground truth dataset.
- 3. Run your system on all 20 websites. Record all findings produced.
- 4. Ask a third-party evaluator to review 5 of the audits without seeing your manual findings.
- 5. Calculate agreement rate: what percentage of AI findings did the manual audit also identify? What percentage were novel but valid? What percentage were incorrect?
- 6. Calculate precision and recall where possible for categories of findings.

19.3 Evaluation Metrics

METRIC	WHAT IT MEASURES	TARGET	HOW TO CALCULATE
Rule-Based Precision	% of rule findings that are correct	>95%	Automated testing
Rule-Based Recall	% of real issues caught by rules	>90%	Manual verification
AI Finding Validity Rate	% of AI findings judged valid by human reviewer	>70%	Human evaluation on 20 websites
AI-Human Agreement	% agreement between AI and manual audit	>60%	Inter-rater reliability
System Processing Time	Average time per audit	<45 seconds	Automated logging
Score Consistency	Score variation on same website run twice	<±5 points	Run same website 3 times, compare
User Task Completion	% of test users who complete full audit	>90%	Usability test with 5 users

19.4 Usability Testing

Conduct a simple usability test with 5 non-technical participants. Ask each to complete a full audit (upload screenshot, fill form, view results) without assistance. Record where

they get confused and whether the report is readable without explanation. Use findings to improve the UI before final demonstration.

SECTION 20

FYP Documentation Requirements

Your FYP documentation is evaluated separately from your working system. Below is the complete structure expected.

CHAPTER	TITLE	KEY CONTENT	DIAGRAMS
1	Introduction	Problem statement, motivation, objectives, project scope, document structure	None
2	Literature Review	Existing tools (Lighthouse, Screaming Frog, PageSpeed, SiteGuru), AI in UX analysis, LLM applications, browser automation research	Comparison table
3	Requirements	Functional requirements (FR1–FR20), non-functional requirements, use cases, user stories	Use case diagram
4	System Design	Architecture, component diagram, data flow, API design, database design, security design, AI pipeline design	Architecture diagram, DFD, ERD, sequence diagrams
5	Implementation	Technology stack justification, development methodology, key implementation decisions, AI prompt design, challenges faced	System screenshots
6	Testing & Evaluation	Test plan, deterministic test results, AI evaluation methodology and results, usability test findings, limitation analysis	Result tables, charts
7	Conclusion	Objectives achieved, limitations, future enhancements, lessons learned	None
App A	API Documentation	All API endpoints, request/response schemas, error codes	None
App B	User Manual	Step-by-step guide with screenshots	UI screenshots
App C	Test Cases	Complete test case table: ID, description, input, expected output, actual output, pass/fail	None

20.1 Documentation Timing

IMPORTANT

Do not leave documentation to the last two weeks. Write each chapter as you complete the corresponding development phase. Write the Literature Review in Week 2. Write the Requirements chapter in Week 3. Write the Design chapter after Phase 1. This prevents the panic documentation sprint that produces poor-quality reports.

SECTION 21

Complete System Workflow

The end-to-end workflow from user action to delivered report. Every step has a technical explanation.

STEP	WHAT HAPPENS TECHNICALLY
1. User Logs In	NextAuth.js validates credentials →JWT issued →session stored in browser.
2. User Submits Audit	Uploads screenshot (PNG/JPG) + optional URL + fills context form.
3. Frontend →Backend API Call	POST /api/audits with multipart form data. Backend creates AUDIT record with status: pending.
4. Screenshot Upload to Cloudinary	Backend sends image to Cloudinary →receives CDN URL →stored in AUDIT.screenshot_url.
5. URL Extraction (if provided)	httpx fetches URL →BS4 parses HTML →text, headings, meta, links extracted →stored temporarily.
6. Deterministic Rule Engine	25 Python rule functions execute against extracted HTML →returns array of rule FINDING objects.
7. AI Vision Call (Claude API)	Screenshot + context →claude-sonnet API →JSON visual findings →validated →stored in FINDING table.
8. AI Copy Call (Claude API)	Extracted text + context →Claude API →JSON copy issues + rewrites →validated →stored in COPY_SUGGESTION.
9. AI Recommendations Call (Claude API)	All findings combined →Claude generates prioritised action plan →JSON →stored.
10. Score Calculation	Formula applied to rule findings + AI finding severity →category scores calculated →stored in SCORE table.
11. Audit Status Updated	AUDIT.status changed to "complete" →Frontend polling detects →Report displayed.
12. Dashboard Renders Results	Findings grouped by category and severity →Score charts →Copy suggestions →Action plan.
13. PDF Export (Optional)	Backend generates HTML report →WeasyPrint converts to PDF →returned as download.

21.1 Failure Handling

Every step must have a defined failure behaviour. URL extraction fails →continue with screenshot-only. AI Vision Call fails →retry once, then mark findings as "Analysis Unavailable" and save partial results. Score calculation fails →display findings without scores. Never leave the user with a blank screen or cryptic error message.

SECTION 22

Final Technical Recommendation

This section does not say "great idea, go build it." It says exactly what you should build, what you should not, what will be hard, and how to succeed.

22.1 What You Should Build

Build a focused, well-executed AI-powered website audit tool that accepts a screenshot and user context, runs three distinct analysis layers (deterministic rules, AI visual analysis, AI copy analysis), and produces a structured professional audit report with documented, reproducible scoring. This is a credible, technically sophisticated, and genuinely useful product that demonstrates real competence in AI integration, web development, and system design.

22.2 What You Should NOT Build in Version 1

- Do not build AI wireframe generation — it requires an entirely different model family and is not achievable.
- Do not build a full headless browser crawler — too complex for a first deployment; add in V2.
- Do not build competitor analysis — out of scope for an FYP.
- Do not build team collaboration features — not academically necessary.
- Do not build an all-in-one prompt that does everything — separate calls per analysis type produces better results.
- Do not build your own accessibility checker — axe-core does this better and you can integrate it in V2.

22.3 Major Risks Ranked

RISK	LIKELIHOOD	IMPACT	MITIGATION
AI findings quality is inconsistent	High	High	Invest heavily in prompt engineering; document variance in FYP
Scope grows uncontrollably	High	High	Lock feature list with supervisor by Week 6; no additions after Week 12
URL scraping fails on JS-heavy sites	High	Medium	Graceful fallback to screenshot-only; document limitation clearly
PDF generation is complex	Medium	Medium	Use HTML template approach with WeasyPrint; defer if time is tight
Authentication bugs delay other work	Medium	Medium	Use NextAuth.js exactly as documented; do not build custom auth
API costs exceed budget	Low	Medium	Implement rate limiting from day one; total cost is well under \$20
Deployment fails at demo time	Low	High	Deploy to production by Week 16; test on live deployment weekly

22.4 What Success Looks Like

At the end of your FYP, you should be able to: (1) open your deployed web application live in a browser; (2) upload a screenshot of any website; (3) fill in context about the business; (4) receive, within 40 seconds, a structured audit report with categorised findings, severity ratings, copy suggestions, a scored dashboard, and an exportable PDF. A supervisor evaluating this should see clear evidence of AI integration, web development competence, system design thinking, and academic rigour in the evaluation methodology.

FINAL VERDICT

This is a technically credible, academically defensible, and practically useful Final Year Project — IF you execute the MVP as defined in this document rather than the full vision document. The core idea is strong. The challenge is discipline: staying in scope, building robustly, testing properly, and documenting rigorously. Two students who do those four things will produce a genuinely impressive FYP.

22.5 Team Accountability

Both students must understand the entire system, not just their assigned half. Your supervisor may ask either student to explain any part of the codebase. Masab should be able to explain how the AI pipeline works. Muzzain should be able to explain how the dashboard renders findings. Weekly knowledge-sharing between team members is not optional — it is how you ensure both graduates can defend the full project.

SECTION APPENDIX

Limitations & Future Enhancements

An honest limitation statement is required in your FYP documentation. Adapt this for Chapter 7.

Known Limitations of the MVP

LIMITATION	TECHNICAL REASON	PLANNED RESOLUTION
URL analysis fails on JavaScript-heavy websites	Simple HTTP fetch cannot execute JS	V2: Playwright headless browser integration
AI findings may vary between runs	LLMs are probabilistic, not deterministic	Document as inherent; improve prompt specificity
Cannot analyse pages requiring authentication	System cannot log into private websites	Out of scope — documented limitation
Visual analysis limited to provided screenshot only	Single image, not full page scroll	V2: Multi-screenshot upload or headless scroll capture
No full WCAG 2.1 accessibility audit	Requires programmatic DOM access	V2: axe-core library integration
No performance measurement (Core Web Vitals)	Requires real browser execution environment	V2: Google PageSpeed Insights API
Cloudflare and bot-protected sites block URL fetching	Bot detection on server-side requests	Screenshot upload is the fallback — by design
PDF generation may fail for very large reports	WeasyPrint memory constraints	Implement pagination in report template

Future Enhancements

- V2 — Playwright Headless Browser: Full JS rendering for URL analysis, multi-viewport screenshots for mobile analysis, automatic scroll-capture for full-page screenshots.
- V2 — Performance Audit: Google PageSpeed Insights API integration for Core Web Vitals (LCP, CLS, FID) without reinventing this solved problem.
- V2 — Full Accessibility Audit: axe-core library integration providing WCAG 2.1 Level AA compliance checking with categorised violations.
- V2 — Audit Comparison: Side-by-side comparison of two audits of the same website to measure improvement over time.
- V3 — Competitor Analysis: Audit multiple websites simultaneously and produce a comparative analysis report.
- V3 — Real-time Monitoring: Scheduled re-audits to detect regressions after website updates.
- V3 — Team Workspace: Multi-user accounts with shared audit history for agency use.
- V3 — Custom Report Branding: White-label PDF reports with client branding for agency output.

This document was prepared as a Technical Feasibility and Project Planning guide for the SiteIQ FYP. It reflects current understanding of the proposed system, identified technical constraints, and realistic scope recommendations as of September 2026. All technology recommendations should be verified against official documentation at the time of implementation. | Confidential — Academic Use Only